



Indian Institute of Information Technology, Nagpur

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

Distributed Computing System

Assignment 1

Under the Guidnace of

Dr. Milind Penurkar

Submitted by

Nimesh Maslekar BT21CSE004

Vikram Bhiwapurkar BT21CSE019

Varad Naik BT21CSE097

Distributed Calculator using .NET Remoting

1) Problem Statement

The purpose of this project is to build a distributed calculator application using .NET Remoting, where a client can interact with a remote server to perform basic mathematical operations. The operations include addition, subtraction, multiplication, and division. The system demonstrates how .NET Remoting can be used to enable communication between a client and a server over a network.

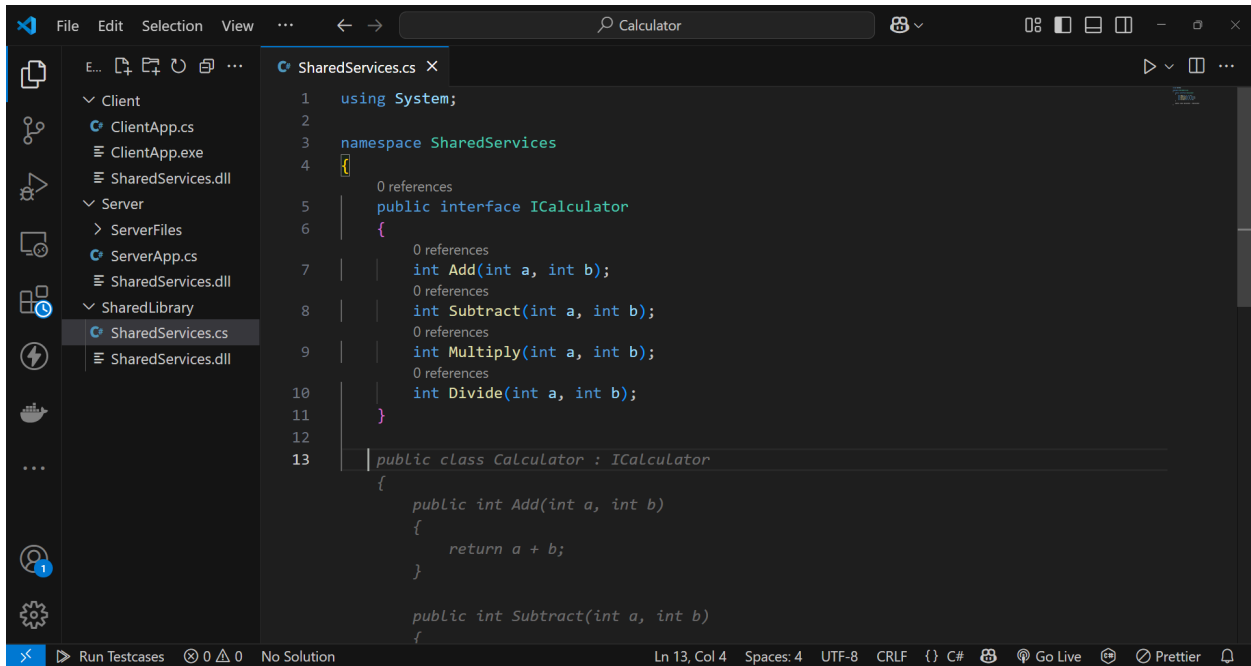
2) Methodology

The project is implemented using the following steps:

- **Step 1: Define the Shared Library:** A shared library (`SharedServices.dll`) is created containing the interface `ICalculator` which defines methods for basic mathematical operations. This interface acts as a contract between the client and the server, ensuring they both adhere to the same communication protocol.
- **Step 2: Implement the Server:** The server application implements the `ICalculator` interface, providing the logic for performing calculations. The server registers the services using .NET Remoting and listens for client requests on a specified port. Thread safety mechanisms are implemented to handle concurrent requests if needed.
- **Step 3: Implement the Client:** The client application connects to the server and interacts with the calculator services. It provides a user interface for the user to select operations and supply operands. The client sends requests to the server and displays the results returned by the server.
- **Step 4: Testing:** Testing involves ensuring all operations (addition, subtraction, multiplication, division) work correctly.

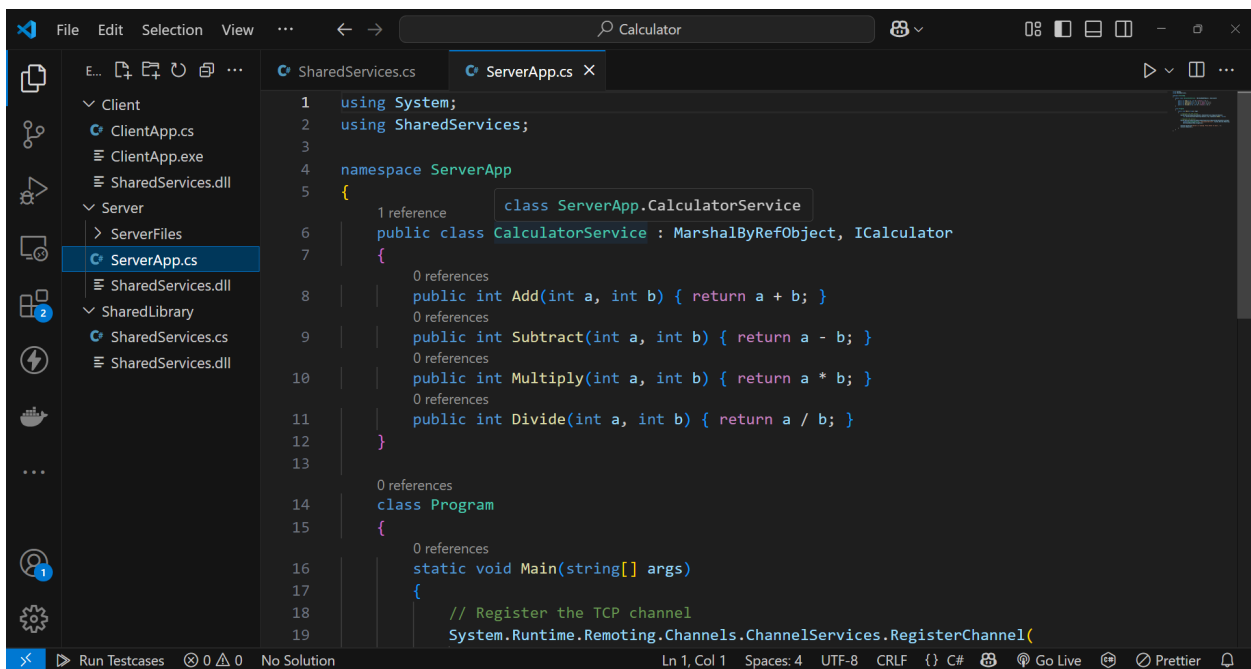
3) Code Snippets

Shared Library (SharedServices.cs):

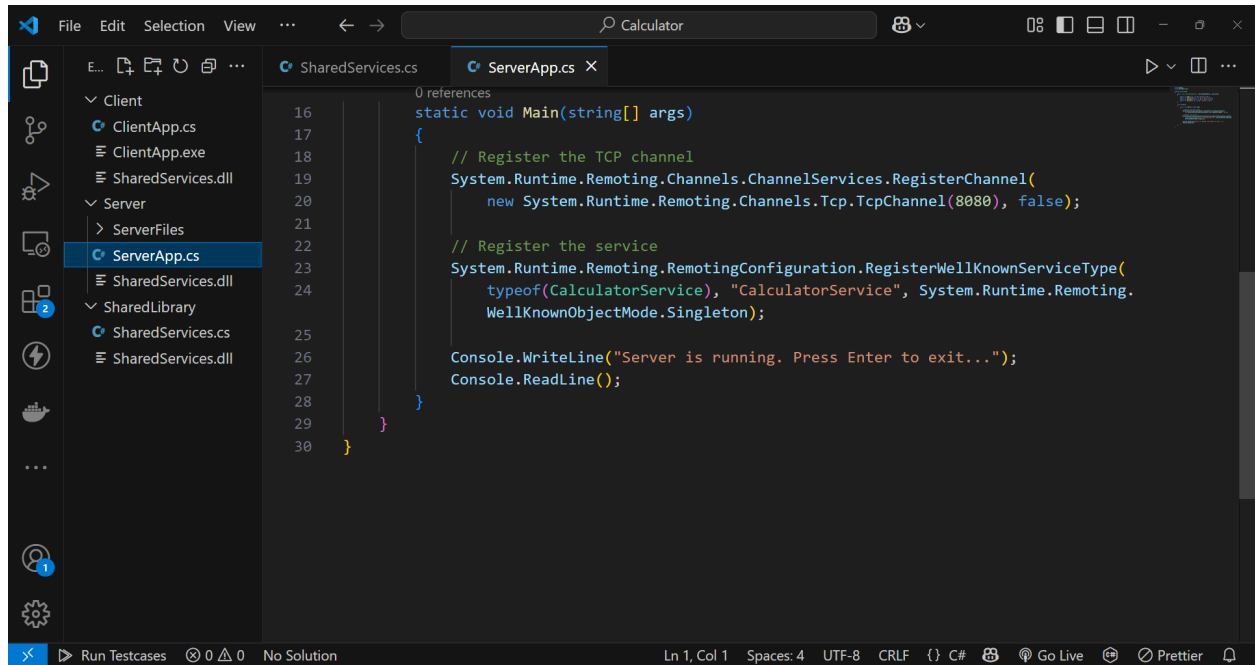


```
1 using System;
2
3 namespace SharedServices
4 {
5     0 references
6     public interface ICalculator
7     {
8         0 references
9         int Add(int a, int b);
10        0 references
11        int Subtract(int a, int b);
12        0 references
13        int Multiply(int a, int b);
14        0 references
15        int Divide(int a, int b);
16    }
17
18    public class Calculator : ICalculator
19    {
20        public int Add(int a, int b)
21        {
22            return a + b;
23        }
24
25        public int Subtract(int a, int b)
26        {
27            return a - b;
28        }
29
30        public int Multiply(int a, int b)
31        {
32            return a * b;
33        }
34
35        public int Divide(int a, int b)
36        {
37            return a / b;
38        }
39    }
40 }
```

Server Implementation (ServerApp.cs):

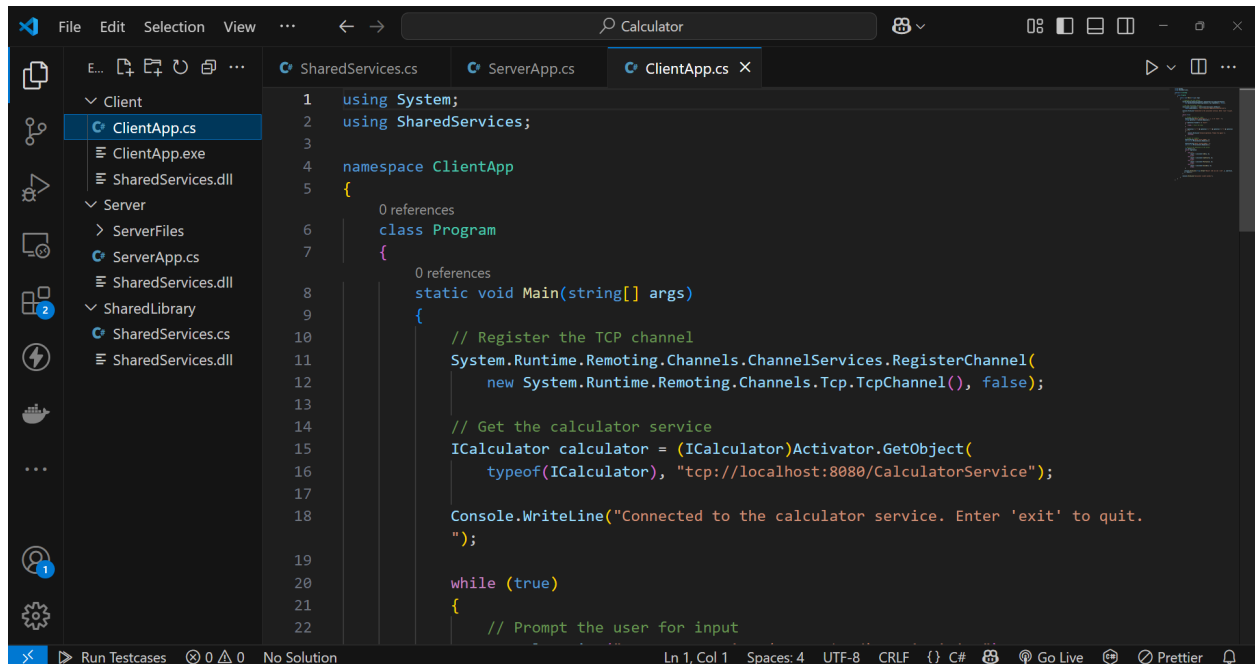


```
1 using System;
2 using SharedServices;
3
4 namespace ServerApp
5 {
6     1 reference
7     class ServerApp.CalculatorService
8
9     public class CalculatorService : MarshalByRefObject, ICalculator
10    {
11        0 references
12        public int Add(int a, int b) { return a + b; }
13        0 references
14        public int Subtract(int a, int b) { return a - b; }
15        0 references
16        public int Multiply(int a, int b) { return a * b; }
17        0 references
18        public int Divide(int a, int b) { return a / b; }
19    }
20
21    0 references
22    class Program
23    {
24        0 references
25        static void Main(string[] args)
26        {
27            // Register the TCP channel
28            System.Runtime.Remoting.Channels.ChannelServices.RegisterChannel(
29                new TcpChannel(), true);
30        }
31    }
32 }
```

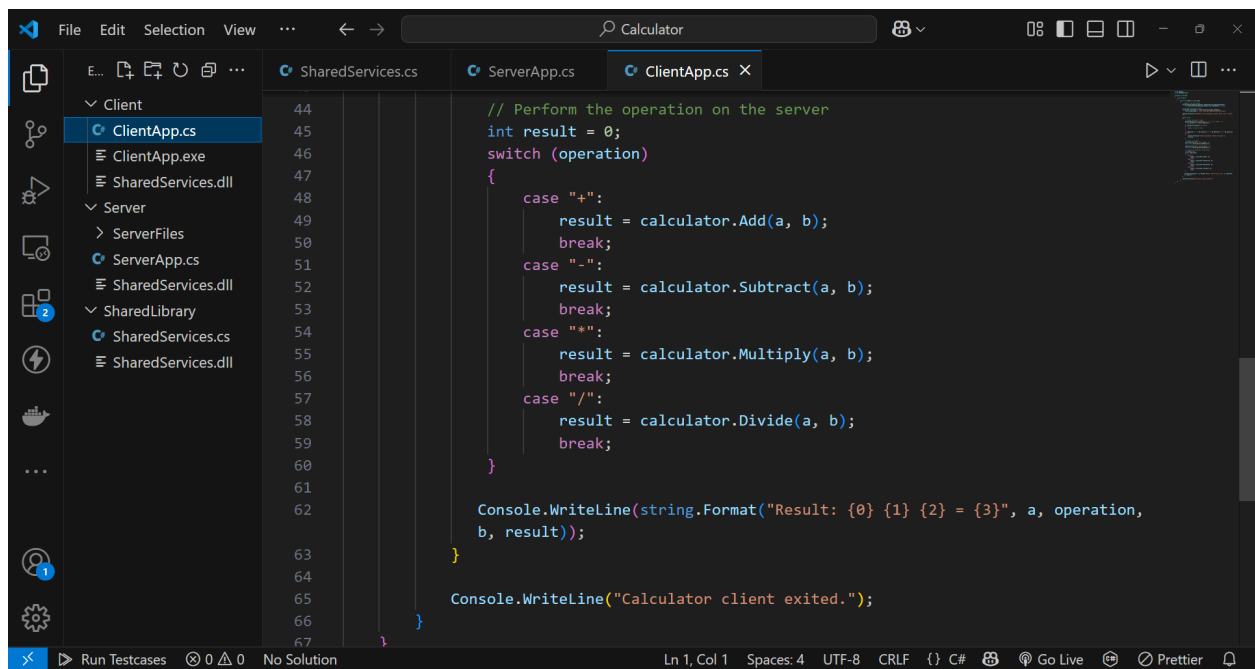
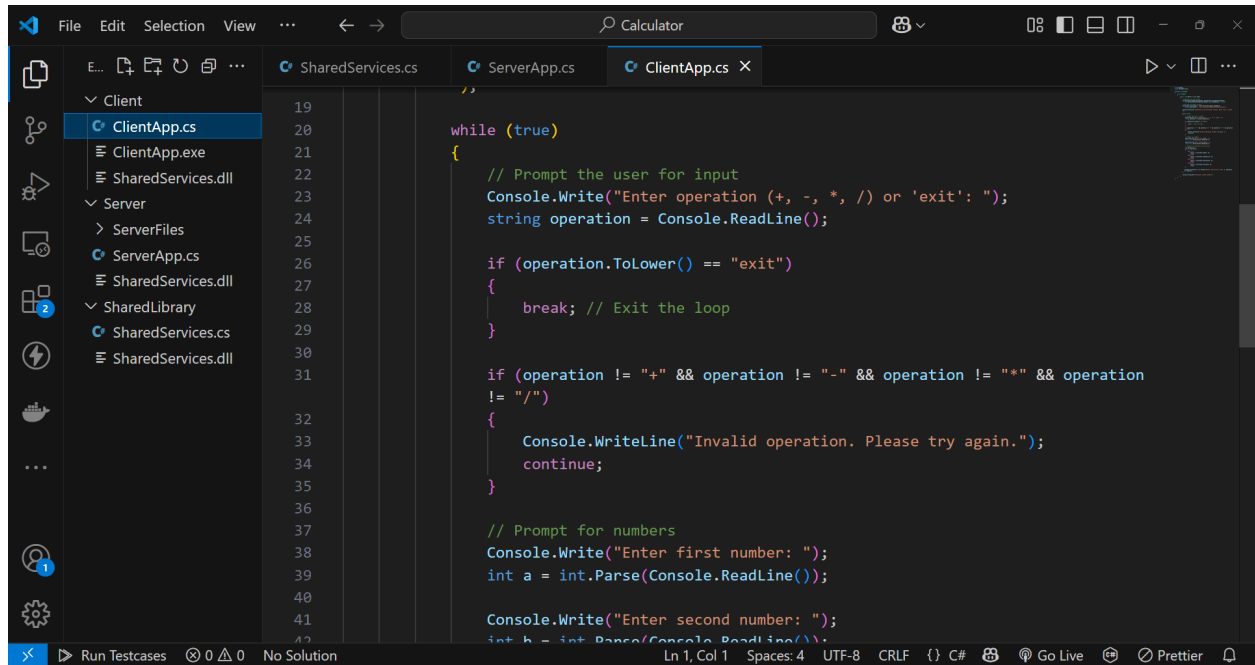


```
16 static void Main(string[] args)
17 {
18     // Register the TCP channel
19     System.Runtime.Remoting.Channels.ChannelServices.RegisterChannel(
20         new System.Runtime.Remoting.Channels.Tcp.TcpChannel(8080), false);
21
22     // Register the service
23     System.Runtime.Remoting.RemotingConfiguration.RegisterWellKnownServiceType(
24         typeof(CalculatorService), "CalculatorService", System.Runtime.Remoting.
25             WellKnownObjectMode.Singleton);
26
27     Console.WriteLine("Server is running. Press Enter to exit...");
28     Console.ReadLine();
29 }
30 }
```

Client Implementation (ClientApp.cs):



```
1 using System;
2 using SharedServices;
3
4 namespace ClientApp
5 {
6     0 references
7     class Program
8     {
9         0 references
10        static void Main(string[] args)
11        {
12            // Register the TCP channel
13            System.Runtime.Remoting.Channels.ChannelServices.RegisterChannel(
14                new System.Runtime.Remoting.Channels.Tcp.TcpChannel(), false);
15
16            // Get the calculator service
17            ICalculator calculator = (ICalculator)Activator.GetObject(
18                typeof(ICalculator), "tcp://localhost:8080/CalculatorService");
19
20            Console.WriteLine("Connected to the calculator service. Enter 'exit' to quit.
21            ");
22
23            while (true)
24            {
25                // Prompt the user for input
26            }
27        }
28    }
29 }
```



4) Method to Run the Project

1. Compile the Shared Library:

- Compile `SharedServices.cs` to create `SharedServices.dll`

2. Run the Server Application:

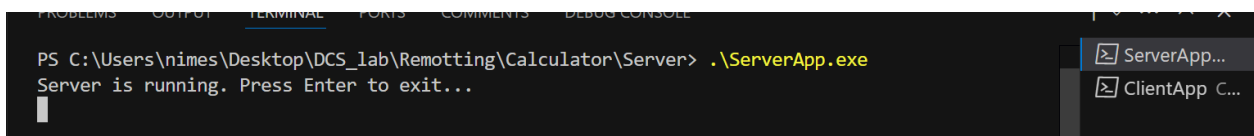
- Compile and run `ServerApp.cs`.
- The server starts listening on a specified port (e.g., `tcp://localhost:8080`)

3. Run the Client Application:

- Compile and run `ClientApp.cs`.
- The client connects to the server and performs operations as requested by the user.

5) Outputs

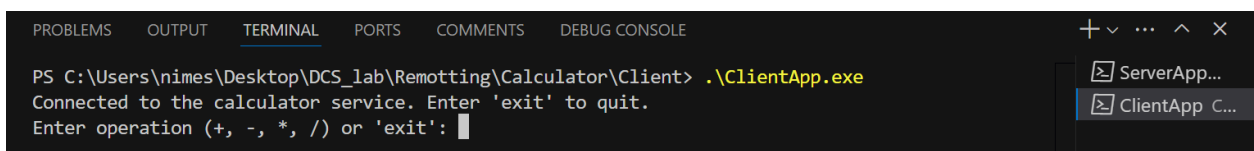
Start the server



```
PS C:\Users\nimes\Desktop\DCS_lab\Remotting\Calculator\Server> .\ServerApp.exe
Server is running. Press Enter to exit...
```

The screenshot shows a Visual Studio terminal window with the 'TERMINAL' tab selected. The command prompt shows the execution of `.\ServerApp.exe` in the directory `C:\Users\nimes\Desktop\DCS_lab\Remotting\Calculator\Server`. The output is 'Server is running. Press Enter to exit...'. On the right side of the terminal, there is a taskbar-like area showing two running applications: 'ServerApp...' and 'ClientApp C...'.

Start the client



```
PS C:\Users\nimes\Desktop\DCS_lab\Remotting\Calculator\Client> .\ClientApp.exe
Connected to the calculator service. Enter 'exit' to quit.
Enter operation (+, -, *, /) or 'exit':
```

The screenshot shows a Visual Studio terminal window with the 'TERMINAL' tab selected. The command prompt shows the execution of `.\ClientApp.exe` in the directory `C:\Users\nimes\Desktop\DCS_lab\Remotting\Calculator\Client`. The output is 'Connected to the calculator service. Enter 'exit' to quit. Enter operation (+, -, *, /) or 'exit':'. On the right side of the terminal, there is a taskbar-like area showing two running applications: 'ServerApp...' and 'ClientApp C...'.

Select the operation

```
Enter operation (+, -, *, /) or 'exit': +  
Enter first number: 74  
Enter second number: 45  
Result: 74 + 45 = 119  
Enter operation (+, -, *, /) or 'exit':
```

```
Enter operation (+, -, *, /) or 'exit': *  
Enter first number: 4  
Enter second number: 6  
Result: 4 * 6 = 24  
Enter operation (+, -, *, /) or 'exit':
```

```
Enter operation (+, -, *, /) or 'exit': -  
Enter first number: 7  
Enter second number: 5  
Result: 7 - 5 = 2
```

```
Enter operation (+, -, *, /) or 'exit': /  
Enter first number: 80  
Enter second number: 5  
Result: 80 / 5 = 16  
Enter operation (+, -, *, /) or 'exit':
```

Exit the client and server

```
Enter operation (+, -, *, /) or 'exit': exit  
Calculator client exited.
```

```
PS C:\Users\nimes\Desktop\DCS_lab\Remotting\Calculator\Client>
```

```
PS C:\Users\nimes\Desktop\DCS_lab\Remotting\Calculator\Server> .\ServerApp.exe  
Server is running. Press Enter to exit...
```

```
PS C:\Users\nimes\Desktop\DCS_lab\Remotting\Calculator\Server>
```

Distributed File Transfer System Using .NET Remoting

1) Problem Statement

The goal of this project is to design and implement a distributed file transfer system using .NET Remoting. The system consists of:

- A server that hosts file transfer services.
- A client that interacts with the server to:
 - List files available on the server.
 - Download files from the server.
 - Upload files to the server

2) Methodology

The project is implemented using the following steps:

Step 1: Define the Shared Library

In this step, a shared library named `SharedServices.dll` is created to define the interfaces for the file transfer services. This library contains the `IFileTransfer` interface which specifies the essential methods for communication between the client and server. The interface includes methods for:

- **Listing files:** Retrieving a list of available files on the server.
- **Downloading files:** Allowing clients to request and download specific files from the server.
- **Uploading files:** Enabling clients to upload files to the server for storage.

The shared library serves as a contract between the client and server, ensuring both sides adhere to the same structure when interacting.

Step 2: Implement the Server

The server application is responsible for hosting the file transfer services defined in the shared library. The implementation of the `IFileTransfer` interface is carried out here. The server performs the following tasks:

- **Registering services:** The server registers the `IFileTransfer` services using .NET Remoting, making them accessible to clients over a network.
- **Listening for requests:** The server listens on a specified port for incoming client requests.

- **Handling file operations:** Provides functionality for listing, downloading, and uploading files.
- **Ensuring thread safety:** Implements mechanisms to handle concurrent requests from multiple clients effectively, preventing data corruption or other concurrency issues.

The server application is designed to run continuously and respond to client requests as they arrive.

Step 3: Implement the Client

The client application is developed to interact with the file transfer services hosted by the server. It uses the `IFileTransfer` interface defined in the shared library to communicate with the server. The client performs the following operations:

- **Connecting to the server:** Establishes a remote connection to the server using .NET Remoting.
- **Providing a user interface:** Presents a menu-driven interface allowing users to choose from available operations (listing, downloading, uploading files).
- **Requesting services:** Sends requests to the server and displays the received data or acknowledgment messages.
- **Handling errors:** Manages exceptions and errors related to network connectivity, file access, or other runtime issues.

The client is designed to be simple and user-friendly, ensuring smooth interaction with the server.

Step 4: Test the System

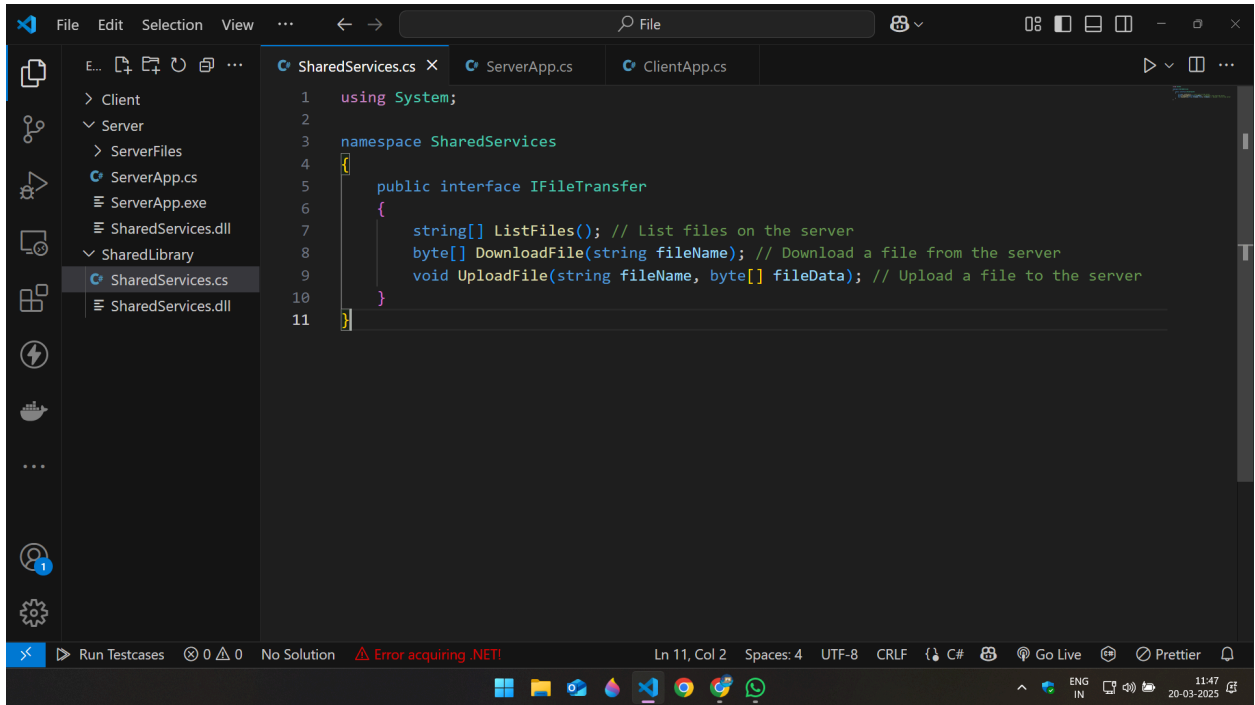
Comprehensive testing is performed to verify the functionality and robustness of the distributed file transfer system. This step involves:

- **Performing file operations:** Testing listing, downloading, and uploading files under various conditions.
- **Testing concurrency:** Ensuring thread safety by simulating concurrent requests from multiple clients to validate the server's ability to handle them without issues.
- **Validating data integrity:** Confirming that files are transferred accurately between the client and server.
- **Checking performance:** Measuring response times and system stability under load.

Testing ensures that the system meets all functional requirements and is capable of handling multiple client requests efficiently.

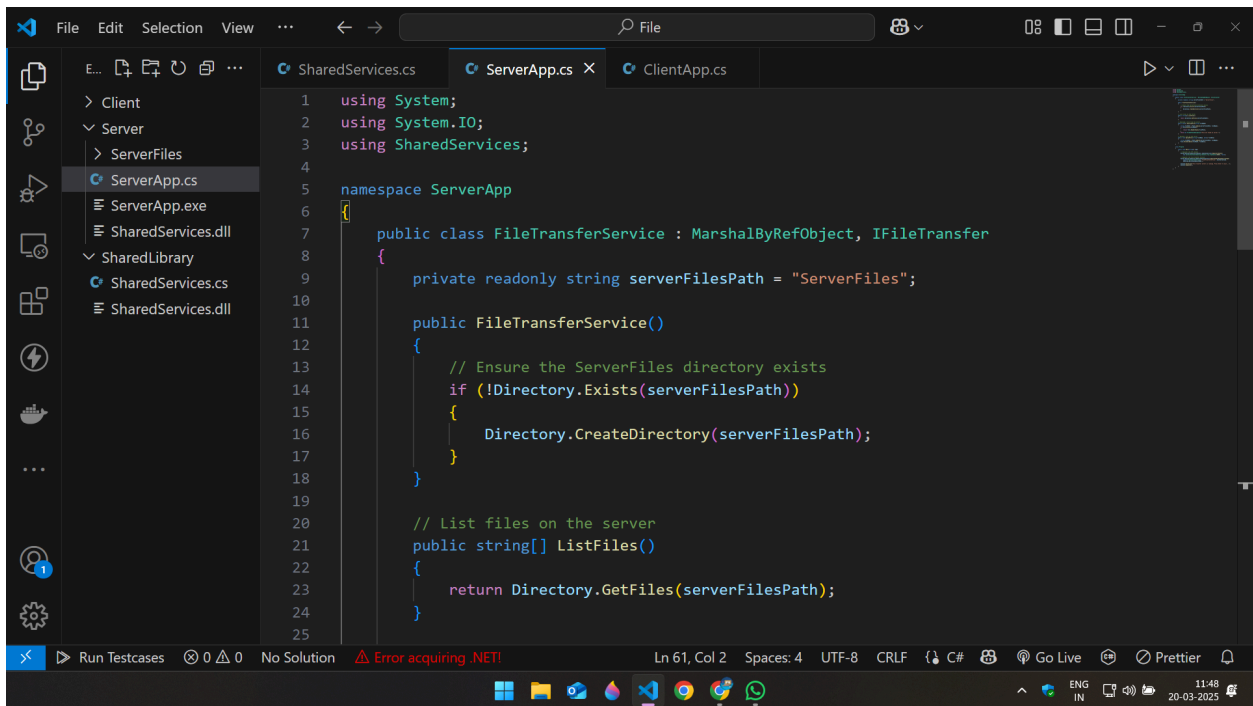
3) Code Snippets

1. SharedServices.cs



```
1 using System;
2
3 namespace SharedServices
4 {
5     public interface IFileTransfer
6     {
7         string[] ListFiles(); // List files on the server
8         byte[] DownloadFile(string fileName); // Download a file from the server
9         void UploadFile(string fileName, byte[] fileData); // Upload a file to the server
10    }
11 }
```

2. ServerApp.cs



```
1 using System;
2 using System.IO;
3 using SharedServices;
4
5 namespace ServerApp
6 {
7     public class FileTransferService : MarshalByRefObject, IFileTransfer
8     {
9         private readonly string serverFilePath = "ServerFiles";
10
11         public FileTransferService()
12         {
13             // Ensure the ServerFiles directory exists
14             if (!Directory.Exists(serverFilePath))
15             {
16                 Directory.CreateDirectory(serverFilePath);
17             }
18         }
19
20         // List files on the server
21         public string[] ListFiles()
22         {
23             return Directory.GetFiles(serverFilePath);
24         }
25     }
26 }
```

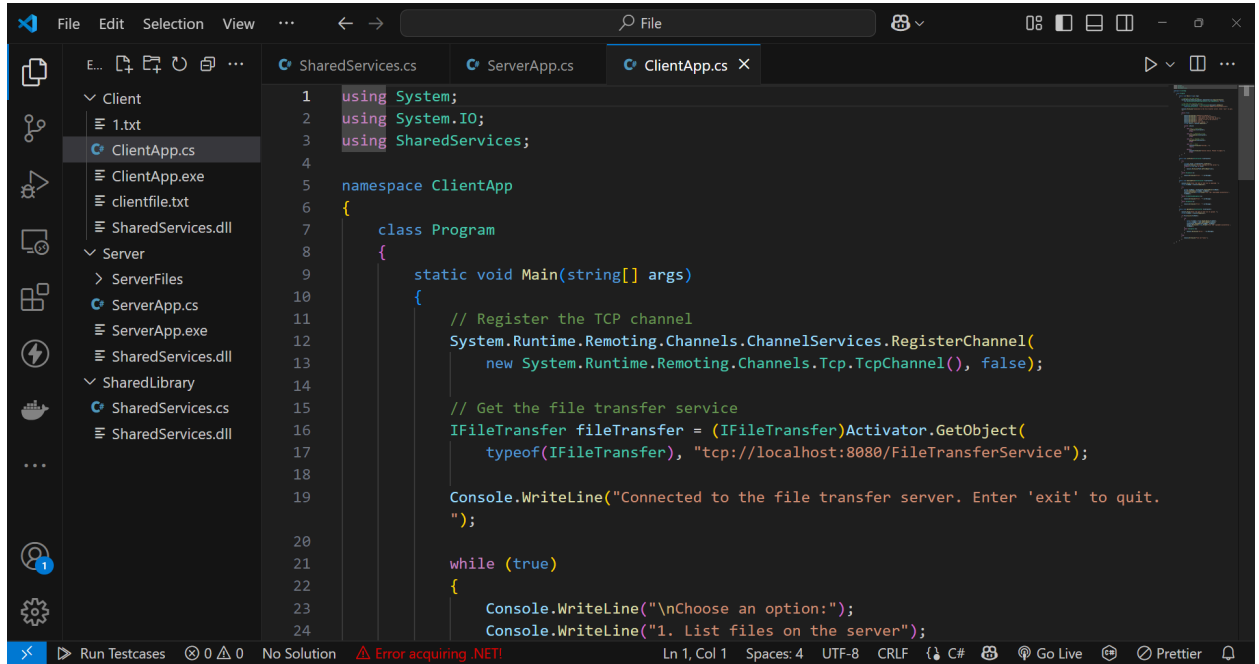
```
25
26 // Download a file from the server
27 public byte[] DownloadFile(string fileName)
28 {
29     string filePath = Path.Combine(serverFilesPath, fileName);
30     if (File.Exists(filePath))
31     {
32         return File.ReadAllBytes(filePath);
33     }
34     throw new FileNotFoundException("File not found on server.");
35 }
36
37 // Upload a file to the server
38 public void UploadFile(string fileName, byte[] fileData)
39 {
40     string filePath = Path.Combine(serverFilesPath, fileName);
41     File.WriteAllBytes(filePath, fileData);
42 }
43
44
45 class Program
46 {
47     static void Main(string[] args)
48     {
49         // Register the TCP channel
```

Run Testcases 0 0 No Solution Error acquiring .NET! Ln 61, Col 2 Spaces: 4 UTF-8 CRLF {} C# Go Live Prettier

```
44
45 class Program
46 {
47     static void Main(string[] args)
48     {
49         // Register the TCP channel
50         System.Runtime.Remoting.Channels.ChannelServices.RegisterChannel(
51             new System.Runtime.Remoting.Channels.Tcp.TcpChannel(8080), false);
52
53         // Register the file transfer service
54         System.Runtime.Remoting.RemotingConfiguration.RegisterWellKnownServiceType(
55             typeof(FileTransferService), "FileTransferService", System.Runtime.
56                 Remoting.WellKnownObjectMode.);
57
58         Console.WriteLine("File transfer server is running. Press Enter to exit...");
59         Console.ReadLine();
60     }
61 }
```

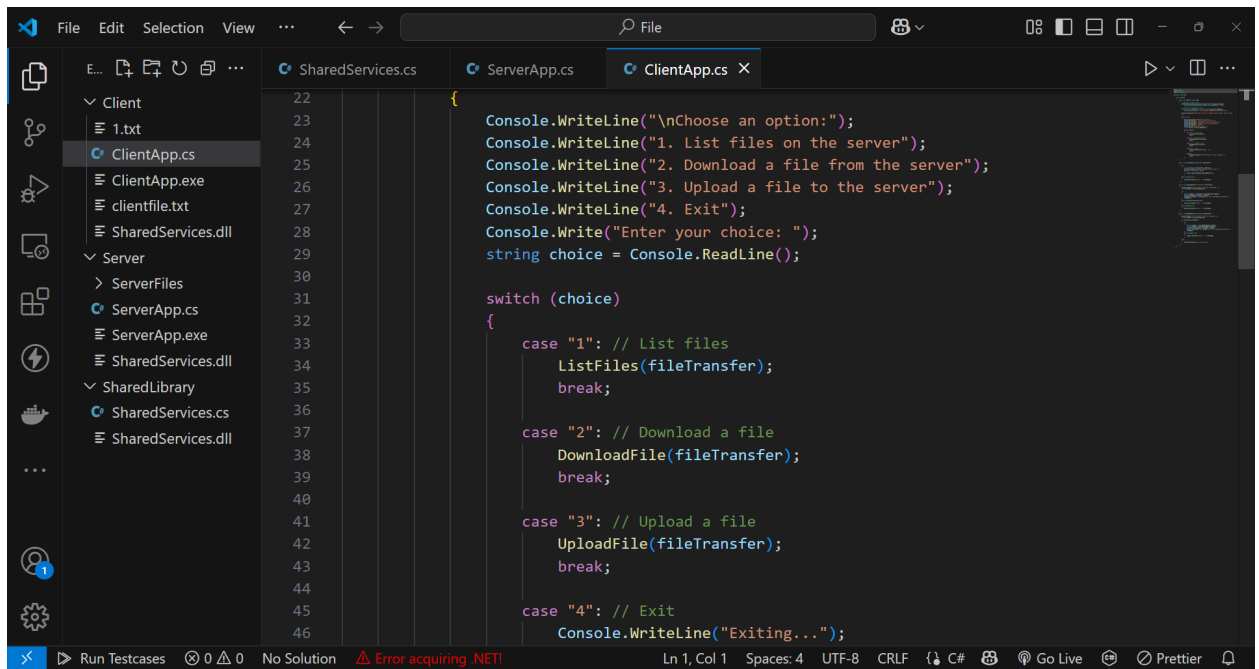
Run Testcases 0 0 No Solution Error acquiring .NET! Ln 58, Col 32 Spaces: 4 UTF-8 CRLF {} C# Go Live Prettier

3. ClientApp.cs



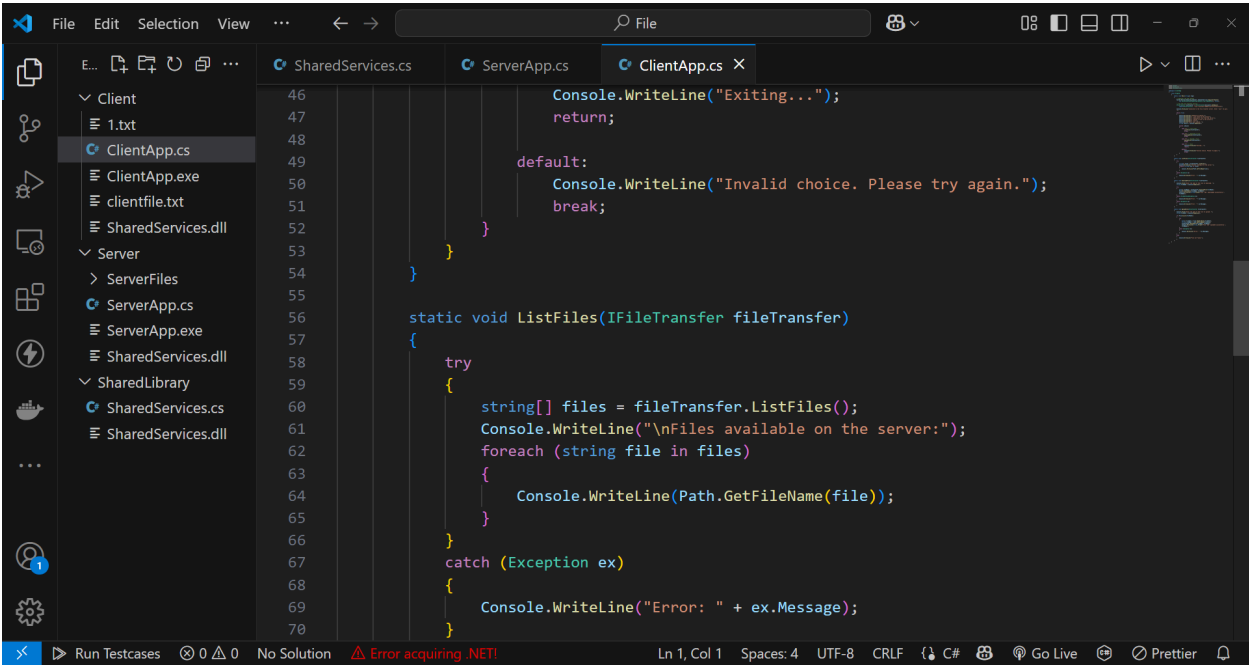
This screenshot shows the initial state of the `ClientApp.cs` file in Visual Studio. The file is open in the editor, and the Solution Explorer on the left shows the project structure. The code defines a `Program` class with a `Main` method that registers a TCP channel, gets a file transfer service, and prints a message.

```
1 using System;
2 using System.IO;
3 using SharedServices;
4
5 namespace ClientApp
6 {
7     class Program
8     {
9         static void Main(string[] args)
10        {
11            // Register the TCP channel
12            System.Runtime.Remoting.Channels.ChannelServices.RegisterChannel(
13                new System.Runtime.Remoting.Channels.Tcp.TcpChannel(), false);
14
15            // Get the file transfer service
16            IFileTransfer fileTransfer = (IFileTransfer)Activator.GetObject(
17                typeof(IFileTransfer), "tcp://localhost:8080/FileTransferService");
18
19            Console.WriteLine("Connected to the file transfer server. Enter 'exit' to quit.
20                ");
21
22            while (true)
23            {
24                Console.WriteLine("\nChoose an option:");
25                Console.WriteLine("1. List files on the server");
```



This screenshot shows the completed `ClientApp.cs` file. The `while` loop has been added, and the `switch` statement handles the user's choice to list files, download a file, upload a file, or exit.

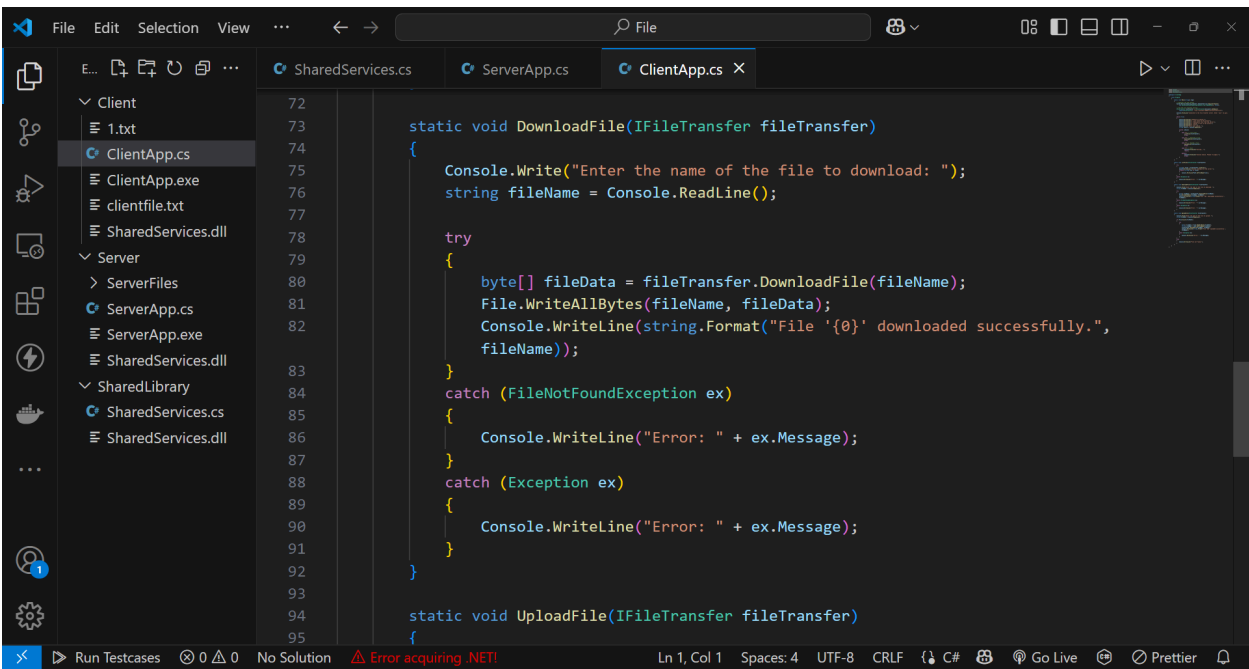
```
22        {
23            Console.WriteLine("\nChoose an option:");
24            Console.WriteLine("1. List files on the server");
25            Console.WriteLine("2. Download a file from the server");
26            Console.WriteLine("3. Upload a file to the server");
27            Console.WriteLine("4. Exit");
28            Console.Write("Enter your choice: ");
29            string choice = Console.ReadLine();
30
31            switch (choice)
32            {
33                case "1": // List files
34                    ListFiles(fileTransfer);
35                    break;
36
37                case "2": // Download a file
38                    DownloadFile(fileTransfer);
39                    break;
40
41                case "3": // Upload a file
42                    UploadFile(fileTransfer);
43                    break;
44
45                case "4": // Exit
46                    Console.WriteLine("Exiting...");
```



This screenshot shows the Visual Studio IDE with the `ClientApp.cs` file open. The file is part of a project with a solution containing `Client`, `Server`, and `SharedLibrary` folders. The `ClientApp.cs` file contains the following code:

```
46 Console.WriteLine("Exiting...");
47 return;
48
49 default:
50 Console.WriteLine("Invalid choice. Please try again.");
51 break;
52 }
53
54 }
55
56 static void ListFiles(IFileTransfer fileTransfer)
57 {
58     try
59     {
60         string[] files = fileTransfer.ListFiles();
61         Console.WriteLine("\nFiles available on the server:");
62         foreach (string file in files)
63         {
64             Console.WriteLine(Path.GetFileName(file));
65         }
66     }
67     catch (Exception ex)
68     {
69         Console.WriteLine("Error: " + ex.Message);
70     }
71 }
```

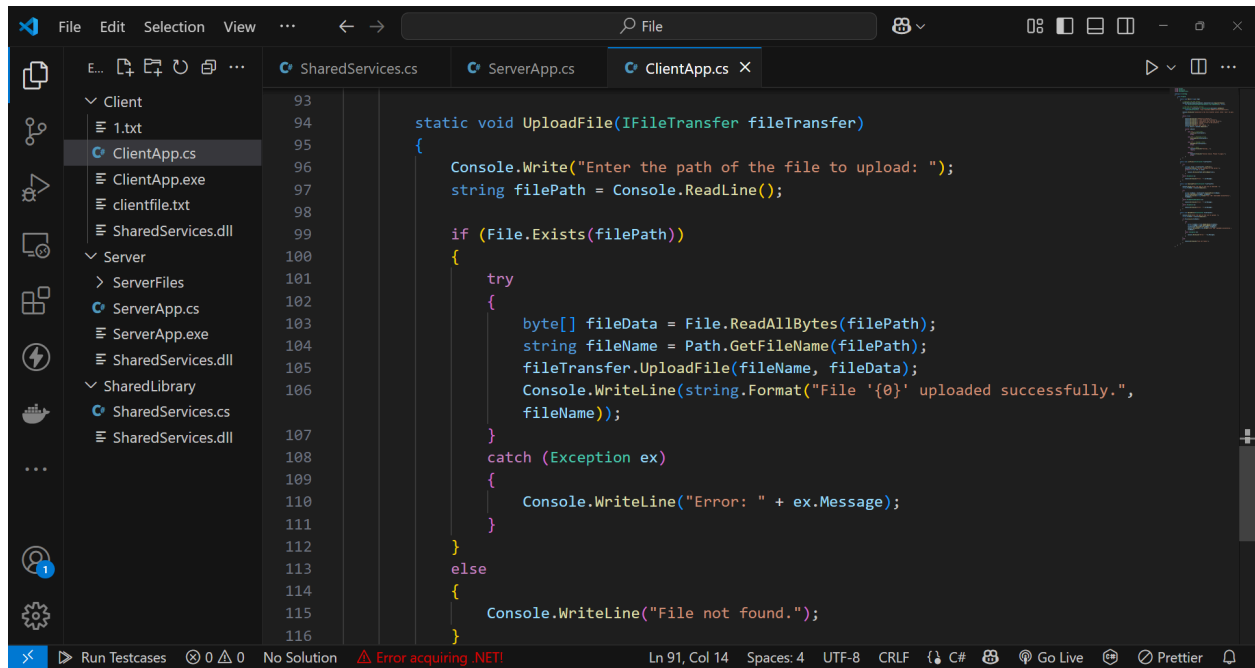
The status bar at the bottom indicates "Ln 1, Col 1", "Spaces: 4", "UTF-8", "CRLF", and "C#". There is a red error icon in the status bar with the message "Error acquiring .NET!".



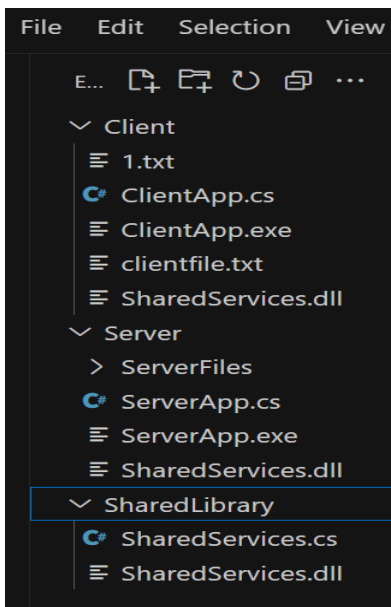
This screenshot shows the Visual Studio IDE with the `ClientApp.cs` file open. The file is part of a project with a solution containing `Client`, `Server`, and `SharedLibrary` folders. The `ClientApp.cs` file contains the following code:

```
72
73
74 static void DownloadFile(IFileTransfer fileTransfer)
75 {
76     Console.WriteLine("Enter the name of the file to download: ");
77     string fileName = Console.ReadLine();
78
79     try
80     {
81         byte[] fileData = fileTransfer.DownloadFile(fileName);
82         File.WriteAllBytes(fileName, fileData);
83         Console.WriteLine(string.Format("File '{0}' downloaded successfully.",
84             fileName));
85     }
86     catch (FileNotFoundException ex)
87     {
88         Console.WriteLine("Error: " + ex.Message);
89     }
90     catch (Exception ex)
91     {
92         Console.WriteLine("Error: " + ex.Message);
93     }
94 }
95
96 static void UploadFile(IFileTransfer fileTransfer)
97 {
98 }
```

The status bar at the bottom indicates "Ln 1, Col 1", "Spaces: 4", "UTF-8", "CRLF", and "C#". There is a red error icon in the status bar with the message "Error acquiring .NET!".



4. Directory Structure

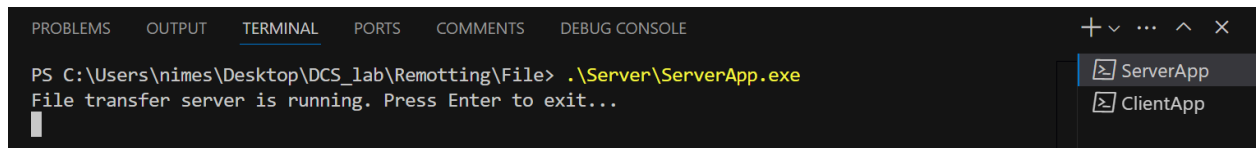


4) Method to run the project

1. Compile the shared library
`csc /target:library SharedServices.cs`
2. Compiler the server
`csc /reference:SharedServices.dll ServerApp.cs`
3. Compile the client
`csc /reference:SharedServices.dll ClientApp.cs`
4. Run the server
`ServerApp.exe`
5. Run the client
`ClientApp.exe`

5) Output

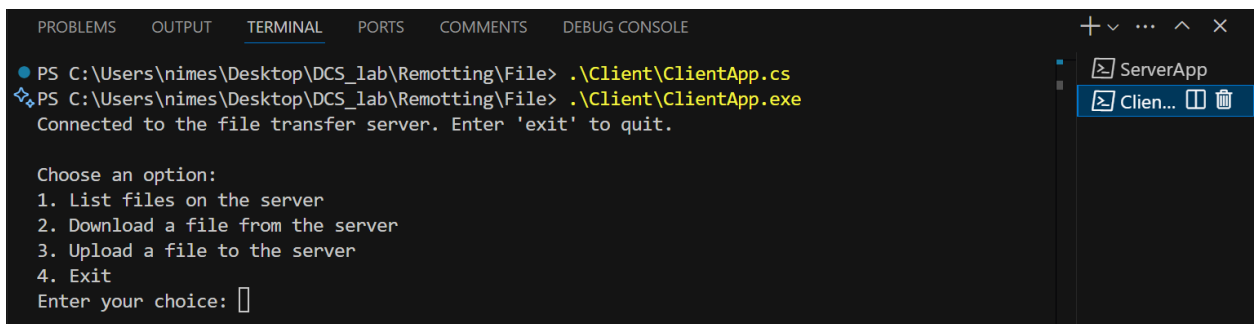
Start the server



The screenshot shows a Visual Studio terminal window with the 'TERMINAL' tab selected. The command prompt shows the execution of `ServerApp.exe` from the directory `C:\Users\nimes\Desktop\DCS_lab\Remotting\File`. The output indicates that the file transfer server is running and prompts the user to press Enter to exit. The taskbar on the right shows icons for `ServerApp` and `ClientApp`.

```
PS C:\Users\nimes\Desktop\DCS_lab\Remotting\File> .\Server\ServerApp.exe
File transfer server is running. Press Enter to exit...
```

Start the client



The screenshot shows a Visual Studio terminal window with the 'TERMINAL' tab selected. The command prompt shows the execution of `ClientApp.exe` from the directory `C:\Users\nimes\Desktop\DCS_lab\Remotting\File`. The output indicates that the client is connected to the file transfer server and prompts the user to enter 'exit' to quit. The taskbar on the right shows icons for `ServerApp` and `ClientApp`.

```
PS C:\Users\nimes\Desktop\DCS_lab\Remotting\File> .\Client\ClientApp.cs
Connected to the file transfer server. Enter 'exit' to quit.

Choose an option:
1. List files on the server
2. Download a file from the server
3. Upload a file to the server
4. Exit
Enter your choice: 
```

Choose a valid option



The screenshot shows a Visual Studio terminal window with the 'TERMINAL' tab selected. The command prompt shows the user entering '1' as their choice. The output displays the files available on the server: `1.txt` and `serverfile.txt`.

```
Enter your choice: 1

Files available on the server:
1.txt
serverfile.txt
```

Download file from server

```
Enter your choice: 2
Enter the name of the file to download: serverfile.txt
File 'serverfile.txt' downloaded successfully.
```

Upload file to the server

```
4. Exit
Enter your choice: 3
Enter the path of the file to upload: to_be_sent_to_server.txt
File 'to_be_sent_to_server.txt' uploaded successfully.
```

Client exits

```
Choose an option:
1. List files on the server
2. Download a file from the server
3. Upload a file to the server
4. Exit
Enter your choice: 4
Exiting...
PS C:\Users\nimes\Desktop\DCS_lab\Remotting\File\Client>
```

Server exits

```
PS C:\Users\nimes\Desktop\DCS_lab\Remotting\File\Server> .\ServerApp.exe
File transfer server is running. Press Enter to exit...

PS C:\Users\nimes\Desktop\DCS_lab\Remotting\File\Server>
```

powershell...

powershell...